

AN ALGORITHM FOR MAZE SEARCHING WITH AZIMUTH INPUT

Vladimir Lumelsky *

Department of Mechanical Engineering
University of Wisconsin
Madison, WI 53706, U.S.A.

Sanjay Tiwari †

Isaac Newton Institute for Mathematical Sciences
University of Cambridge
Cambridge CB3 0EH, U.K.

Abstract—The performance of systems for sensor-based robot motion planning is known to depend heavily on the basic search algorithms used. Typically, these are provable graph search schemes modified into maze search algorithms by taking into account the topology of the physical objects in the workspace – for example, the fact that the boundary of a 2-dimensional object is a simple closed curve. Although several maze search algorithms are already available, more are needed to improve performance and fill gaps in required properties. This work presents a new algorithm that possesses properties not covered so far, and then assesses its performance. The algorithm allows one to use directional input information (such as azimuth angles) rather than the usual cartesian coordinates; it also allows the robot to “cut corners” on its way to the target, thus shortening the path. It will be of special value for applications to mobile robots, where obtaining cartesian coordinates is prone to quick error accumulation.

I. INTRODUCTION

We consider the problem of motion planning for a point automaton (a robot) operating in a planar scene with unknown obstacles. Continuous (on-line) planning is assumed, based on information from the automaton’s sensors. Only provable (nonheuristic) approaches are considered. The subject of this work is to present a new motion planning algorithm which fills an important gap in prior work. The algorithm presented here accepts input information in terms of directions, or angles, such as from a compass – as opposed to requiring knowledge of (the automaton’s) cartesian coordinates; it also allows the automaton to walk directly toward its target position when physically feasible (except in some special cases). No existing motion planning strategies have these properties.

The objective of the automaton is to generate a collision-free path from a starting point S to a target

point T , or to prove that T cannot be reached from S if true; T is distinct from S . We assume that each obstacle’s boundary in the automaton’s environment is a simple, piecewise smooth closed curve, of finite perimeter and arbitrary shape, and this curve has a finite number of inflection points and corners. Notice that this is a very general assumption and is satisfied by all obstacles encountered in robotics. If two obstacles touch each other, they are considered one. The automaton has no knowledge of the obstacles’ geometry or positions. Its sensors allow it to detect an obstacle once it approaches one, and to walk along its boundary. Although a variety of sensors can be considered (e.g. vision), we assume for convenience (and, it can be shown, without loss of generality) that the robot has simple tactile sensors: that is, the automaton knows about an obstacle only when it touches it. To emphasize the generality, the simulation at the end of the paper shows the use of vision.

By its very nature, this model calls for on-line (real-time) algorithms; that is, the automaton plans its motion “on the fly”, based on its prior motion and the latest sensor data. The model is known [4] as *planning with incomplete information*, or *motion planning with uncertainty*, or *sensor-based motion planning*¹. Algorithmically, the problem of sensor-based motion planning is one of *maze searching* [1]. A maze is often described as a finite collection of narrow corridors interconnected in different ways, with clearly defined intersections and perhaps, disconnected “islands”. On its way to the target the automaton will have to cross – say, along straight lines – the free areas between obstacles. These *free path segments*, together with the corresponding pieces of obstacle boundaries, form a graph whose vertices are points S , T , and the intersection points between the free path segments and the obstacles. The maze-to-graph reduction suggests that the algorithm analysis can be done either in the graph domain or the

*Supported in part by NSF Grant IRI-9220782 and DOE (Scandia Laboratories) Grant 18-4369C.

†Supported in part by SERC (U.K.) Grant GR-G59981

¹This is distinct from the model referred to as *planning with complete information*, in which full information about the geometry and positions of objects in the scene is given. A survey of work in this area can be found in [2], [3].

maze domain. Since different motion planning strategies generate free path segments differently, the maze-graph relation is not unique.²

All existing motion planning algorithms are *metric* – they rely on the robot's knowledge of its own and the target's cartesian coordinates, and on measuring the distances it traverses. The predominant technique for finding the current location of a mobile robot is the so-called dead reckoning – that is, the robot's current location is found by integrating the motion of its wheels, similar to the automobile tachometer. Measuring the distance traversed is done similarly. Given the realities of a robot's life – uneven floors, changing friction between wheels and the ground, frequent turns etc. – the accuracy of that integral is low and decreasing with time, and so dead reckoning is inherently prone to quick error accumulation.

This makes metric algorithms brittle – the robot can simply get lost. On the other hand, angular input information – say, from a compass or radar – gives global coordinates and so is not prone to error accumulation. This suggests that *nonmetric* algorithms that would accept angular input information and would not rely on measuring distances would be more desirable in practice. No such algorithms exist, however. The only provable nonmetric algorithm available, called the *Pledge algorithm* [6], can help escape from a finite maze but cannot be used for reaching a specific point in the maze. The *Angulus* algorithm, stated and proven in Section II, is the first nonmetric approach to the task at hand. Also considered in this paper, and incorporated in the algorithm, are conditions under which the automaton can walk directly toward the target, without jeopardizing convergence. Clearly, straight-line motion towards the target, whenever physically possible, is preferable. It is not allowed, however, in the existing algorithms.

II. THE ANGULUS ALGORITHM

The *Angulus* algorithm uses, as an essential piece of its input information, the notion of two *cumulative turning angles*: of the automaton's position vector s with respect to the line ST , and its velocity vector v with respect to ST . There are two counters, α and β , which store the current values of these angles, respectively, at all times. The cumulative turning angles are defined as follows (Figure 1): When the robot moves along the obstacle boundary, the counter reading $\alpha_i = \alpha(C_i)$ at any point C_i is the cumulative angle made by the line C_iT (that is by vector $s_i = s(C_i)$) with respect to the line ST , with T taken to be the origin and the initial value of α_i at S is taken to be zero. The counter reading $\beta_i = \beta(C_i)$ is the cumulative angle made by the vector $v_i = v(C_i)$ with respect to

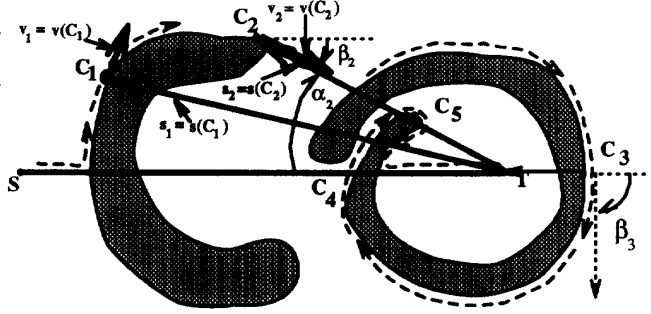


Fig. 1. The cumulative turning angles.

the line ST , with the initial value of β_i at S (v_i pointing towards T) taken to be zero, see Figure 1; the dashed line illustrates the robot path from S to T . Notice that α measures the rotation of the point C around the point T , whereas β measures the rotation of the vector v in the plane, irrespective of T . Since we measure angles counter-clockwise, the counter readings at points $C_2, \dots, C_5$ on the obstacle boundaries are given by: $\alpha_1 = -\pi/12$, while $\beta_1 = 5\pi/12$, $\alpha_2 = \beta_2 = -\pi/6$, $\alpha_3 = -\pi$ while $\beta_3 = -\pi/2$, $\alpha_4 = -2\pi$ while $\beta_4 = -3\pi/2$, and $\alpha_5 = \beta_5 = -13/6\pi$.

The term *cumulative* refers to the fact that the angle is set to zero only once, at the starting point S , and is then integrated along the path, taking into account its sign, with the counter-clockwise direction being positive. For example, if the line ST were parallel to the North-South axis, then one could use a compass to measure the cumulative turning angle β_i , which would then be the total angle through which the compass needle has moved with respect to a fixed mark on the compass circumference – the mark always pointing in the direction of the velocity vector v_i . Notice that by introducing the azimuth angle, β_i can always be measured by a compass; therefore, without loss of generality, we can assume that ST lies along the North-South direction. The angle α_i cannot be measured by a compass, but if there is a radio beacon (or similar device) at the target T , then the robot can calculate α_i provided it can detect the direction of the signal. The *Angulus* algorithm operates as follows (assume that at a hit point the robot turns left and passes around an obstacle clockwise; C_i is the current position, v_i the velocity vector at C_i , and T the target point).

The Angulus algorithm.

1. Proceed along the line ST ; if the target is reached, stop.
2. If an obstacle is encountered at a hit point H_j , turn left and follow the obstacle boundary, while simultaneously measuring the cumulative angles α and β .

² An interesting discussion on the subject can be found in [5].

3. At a point C_i on the obstacle boundary, if it is physically possible to walk straight toward T , and the current velocity vector v_i points toward T , and both $\alpha_i, \beta_i \in (-2\pi, 0)$, proceed directly toward T . If T is reached, stop; if an obstacle is encountered, go to Step 2.
4. If it occurs at least once that $\alpha \leq -2\pi$, then leave obstacle for T only when $\alpha_i = \beta_i = -2\pi$. (Notice that at that moment in general the directions of s and v are not the same. If these directions are indeed different, β_i is measured after first turning counter-clockwise toward T .) If T is reached, stop; if an obstacle is encountered, go to Step 2.
5. If a hit point H_j is encountered twice without first leaving the obstacle, the target is unreachable.

In Step 5 of the algorithm, reachability is identical with the target lying outside all obstacles i.e. in the set-theoretic exterior of the set of all obstacles. The notion of inside, outside and reachability can be extended to case of obstacles with holes, but in the interests of simplicity we avoid that. The following theorem shows that the algorithm is provable under a finiteness condition.

Theorem 1 *The Angulus algorithm converges if and only if the workspace has a finite number of obstacles.*

Proof: The proof of sufficiency demonstrates that, in an environment with a finite number of obstacles, infinite cycles cannot occur in the path of the robot. The necessity is then shown via a counterexample. Infinite cycles can occur in two different ways [7]: the robot can keep going around the boundary of an obstacle an infinite number of times without leaving it, or it can leave an obstacle at the same leave point an infinite number of times. In order to rule out infinite cycles, the following two facts about the algorithm will be proven below:

1. If the target is reachable, then the robot cannot return to a hit point on the obstacle before first leaving the obstacle. (In fact, if the robot returns to the same hit point before first leaving the obstacle, it decides that the target is unreachable by Step 5.)
2. The robot cannot leave from the same leave point twice.

Proof of Fact 1: When the robot encounters an obstacle at hit point H_j , the counter reading is either (a) $\alpha_j = \beta_j = -2\pi$, or (b) $\alpha_j = \beta_j \in (-2\pi, 0)$. For case (a), let the line $\overrightarrow{H_j T}$ next cross the obstacle boundary at point $\bar{L}_j$ (Figure 2a). Note that $\bar{L}_j$ is not necessarily a leave point and is, in general, different from L_j - the next

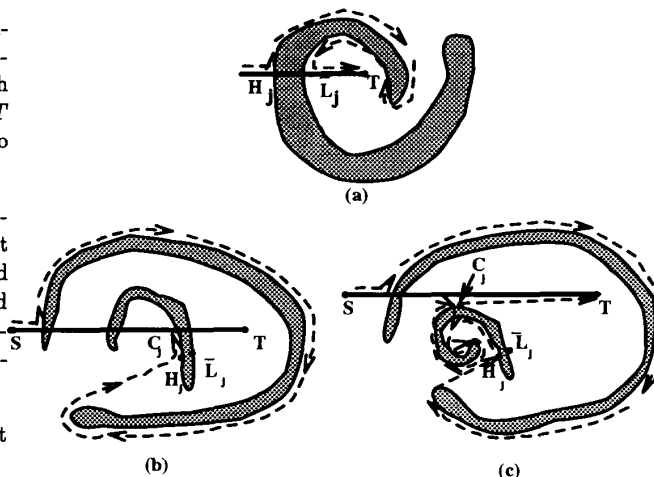


Fig. 2. Illustration for the proof of Fact 1

leave point after H_j . If the robot has not left the obstacle before it reaches $\bar{L}_j$, then it will certainly leave at $\bar{L}_j$, i.e. $\bar{L}_j = L_j$: this is because at $\bar{L}_j$, $\alpha_i = \beta_i = -2\pi$ (follows from elementary winding number considerations). For case (b), let $\bar{L}_j$ be the next point of intersection of the directed line $\overrightarrow{H_j T}$ with the obstacle boundary. If, while moving from H_j to $\bar{L}_j$, the robot goes through a point C_j where the counter reading is $\alpha_j = -2\pi$ (Figure 2b), then the motion of the robot from C_j onwards is identical to the one in which the robot first moves along the line ST with counter reading -2π and encounters the obstacle at C_j . This has already been settled in case (a). If the obstacle boundary segment between H_j and $\bar{L}_j$ does not cross the line segment ST , then there is a point C_j on the obstacle boundary such that the tangent line at C_j passes through T and $\alpha_j = \beta_j \in (-2\pi, 0)$ (Figure 2c). This follows from Rolle's theorem or Mean-value theorem of calculus [9] applied to the function $\alpha_j - \beta_j$ along the obstacle boundary from H_j to $\bar{L}_j$, since even at singular points on the obstacle boundary we may assume that the robot changes directions smoothly. Thus the robot cannot return to H_j without first leaving the boundary.

Proof of Fact 2: Suppose that the robot leaves twice from a leave point L_j . This means that at L_j either $\alpha_j = \beta_j \in (-2\pi, 0)$ both times and the path of the robot does not cross the line ST or $\alpha_j = \beta_j = -2\pi$, both times. Winding number considerations imply that there exists at least one counter-clockwise subcycle inside the cycle at L_j . Let the first of these subcycles begin at Q . There is only one way in which such a subcycle can be formed:

The robot forms a subcycle of the type shown in Figure 3 which involves one straight line movement $\overrightarrow{QR}$ to-

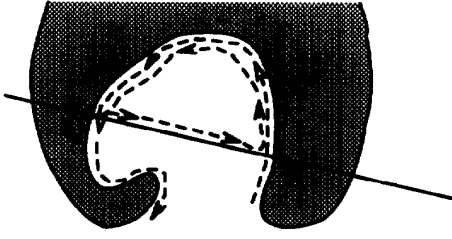


Fig. 3. Illustration for the proof of Fact 2 - the formation of a subcycle

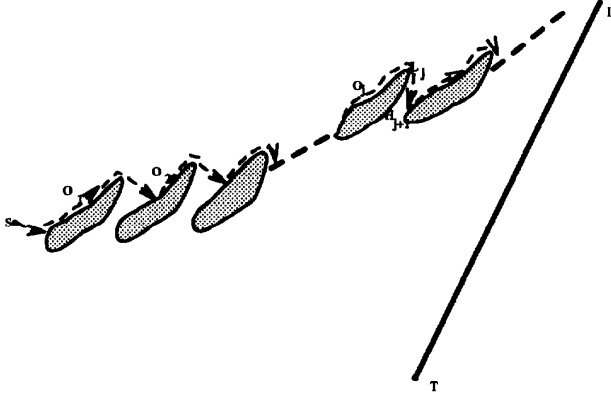


Fig. 4. Illustration for Example 1

ward the target.

1. If $\overrightarrow{QR}$ is not a subset of the line ST , then at R (the first time) $\alpha_j(R) = \beta_j(R) > -2\pi$. But this implies that the robot will not be able to leave at Q in the direction $\overrightarrow{QR}$, since $\beta_j(Q) > 0$.
2. If $\overrightarrow{QR}$ is a subset of the line ST , then at R , $\alpha_j(R) = -2\pi$, $\beta_j(R) = -4\pi$. (Notice that T cannot lie inside the subcycle). We claim that this configuration of the two angles is not possible for the following reasons: Starting from $\alpha_j = \beta_j = -2\pi$, β_j has to change directions at least once, with respect to ST , to arrive at $\beta_j = -4\pi$. Clearly this cannot happen unless the path crosses ST . But the path cannot cross ST , because it is forced to leave along ST towards T , by the algorithm. The only way it can cross ST is if T is surrounded by the path in which case α_j will decrease to -4π as well. Thus the robot will not be able to leave L_j a second time.

Since we have assumed that the boundary of any obstacle has a finite number of inflection points (including corners), it can be easily seen that the number of points

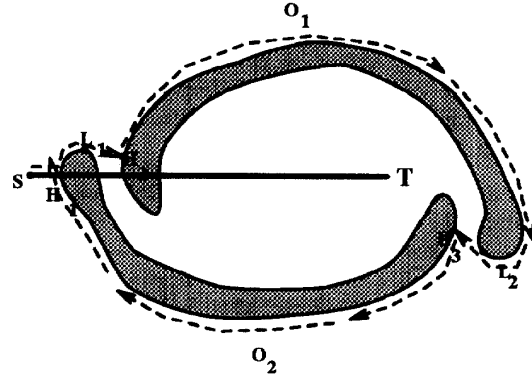


Fig. 5. Illustration for Example 2: The counter cannot be reset to zero at a hit point.

on the obstacle boundary where the tangent lines pass through T is also finite. Since, by assumption, the number of obstacles in workspace is finite, the set of all possible leave points in the robot's path is also finite. Facts 1 and 2 (proved above) now imply that our algorithm is convergent. *qed*.

To show the necessary condition of Theorem 1, consider the following example:

Example 1 Consider the infinite sequence of obstacles $\{O_j\}$ in Figure 4. The robot starts at S and moves along a part of the boundary of each obstacle O_j , between H_j and L_j . As $j \rightarrow \infty$, the line L_jH_{j+1} becomes asymptotically parallel to the line LT and $d(L_j, T) \rightarrow \infty$. Thus the Angulus algorithm does not converge for an infinite number of obstacles in the workspace.

In Step 2 of Angulus, the counter reading α_i refers to the cumulative turning angle at C_i . As remarked earlier, this means that when the robot encounters a new hit point the counter is not reset to zero. In combination with Step 4 of the algorithm, this suggests that if for whatever reason the robot accumulates a large negative angle $\alpha \leq -2k\pi$, $k \geq 1$, then there will be a long unpleasant "unwinding" of α , before the robot can leave toward T . However, an attempt to fix this by resetting the counter to zero in Step 2 at each new hit point would result in the loss of convergence, as the following example demonstrates. (The example was provided by K. Harinarayan.)

Example 2 The workspace has two obstacles O_1 and O_2 enveloping the target T , as shown in Figure 5. If one resets the counter to zero at each new hit point, one gets an infinite, with the leave points alternating, ad infinitum, between L_1 and L_2 . Note that this is equivalent to not putting any conditions on the turning angle.

In Step 3 of the algorithm, if any time $\alpha(C)$ reaches the value -2π , then the robot can only leave at points L where $\alpha(L) = \beta(L) = -2\pi$. Note that for any position C of the robot $\alpha(C) \leq \beta(C)$. This means that, if any time the angle α becomes less than or equal to -2π , the direct motion towards the target T ceases from that point onwards. One would like to have a more permissible condition for leaving an obstacle – for example the following: The robot leaves an obstacle at L if the vector v points towards T and if $\alpha(L) \in (-2\pi, 0)$. This condition would, in effect, put conditions only on α . Similarly one could ask for a condition only in terms of β . For example: the robot leaves an obstacle at a point C if and only if $\beta(C) \in (-2\pi, 0)$ and the vector v points towards T . Unfortunately, neither of these conditions work (for counterexamples see [10]).

We now show that the algorithm, *Pledge* [6], which is designed to escape from a finite maze, can be considered to be a special case of the *Angulus* algorithm.

Consider a maze in the plane and a starting point S inside the maze. Map the maze to a corresponding maze on the Riemann sphere, by the usual identification of the plane with the sphere via stereographic projection (Figure 6a), see [11], pages 124-125. Escaping the planar maze then becomes equivalent to the problem of reaching the North Pole on the sphere. Stereographically project the Riemann sphere again onto the horizontal plane through the North Pole N . The target point T is mapped to the origin and the starting point S to the point at infinity. Apply *Angulus* to this maze (with local direction “right” and the interval $(0, 2\pi)$ replacing the interval $(-2\pi, 0)$ in steps 3 and 4), and designate the initial longitude (the line one moves along) to be the M-line. Translating back to the plane, the local direction will change back to “left” but the interval $(0, 2\pi)$ will remain as is, and give the following algorithm (Figure 6b):

1. Pick any point O in the plane to be the origin and consider the family of radial lines originating at O .
2. At S , designate any outward radial line as the M-line; move along M-line.
3. When an obstacle is encountered, move left along the obstacle boundary, while measuring the cumulative angles α and β , where α is the cumulative turning angle of the position vector with respect to the origin O and β , the cumulative turning angle of the velocity vector with respect to the M-line.
4. At a point L on the obstacle boundary, if it is physically possible to walk outwards along a radial line and the current velocity vector v_i points radially outwards, and $\alpha_i = \beta_i \in (0, 2\pi)$, proceed in that radial direction.

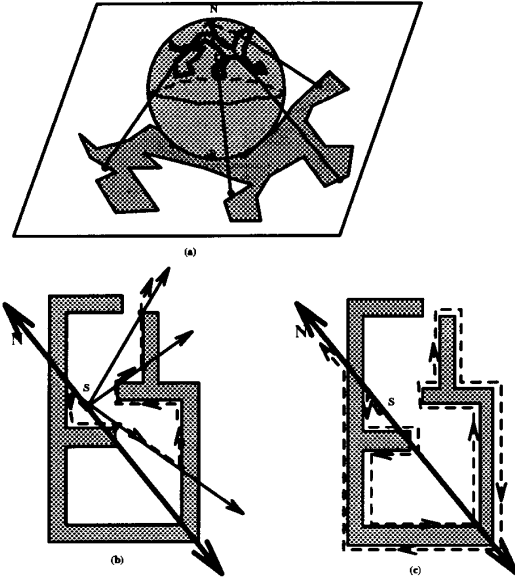


Fig. 6. (a) The *Angulus* algorithm on the Riemann sphere, (b) escaping a maze – the path of the *Angulus* algorithm, (c) the path of the *Pledge* algorithm; S is the starting point, N the direction to the North.

5. If, at least once, $\alpha \geq 2\pi$, then at point L one can leave the obstacle along a radial direction only when $\alpha_i = \beta_i = 2\pi$.

If one takes $S = O$, then all movement (until $\alpha = 2\pi$) will simply be radially away from S . On the other hand, if O is taken as a point at infinity then the *Angulus* algorithm becomes the *Pledge* algorithm. Figures 6b and 6c compare the performance of *Angulus* with that of the *Pledge* algorithm. Notice that the paths of the *Angulus* algorithm are more natural.

The following theorem shows that the path length performance of the *Angulus* algorithm is similar to that of *Pledge*, *Bug 2* [4] and some other algorithms.

Theorem 2 For any given $\delta > 0$ (however small), $n > 0$ (however large) and any given finite perimeters p_i of the obstacles and distance D between S and T , there exists a set of obstacles for which the path length P satisfies:

$$P \geq D + n \sum p_i - \delta. \quad (1)$$

Proof: We first show that (1) is true for one obstacle. Note that a spiral obstacle can be easily built, Figure 7, such that under *Angulus*, one segment of the obstacle perimeter – p^n (thick solid line) – will be traversed n times. Similarly another segment, say p^{n-1} , will be traversed $(n-1)$ times, etc. Let D , as before, be the total

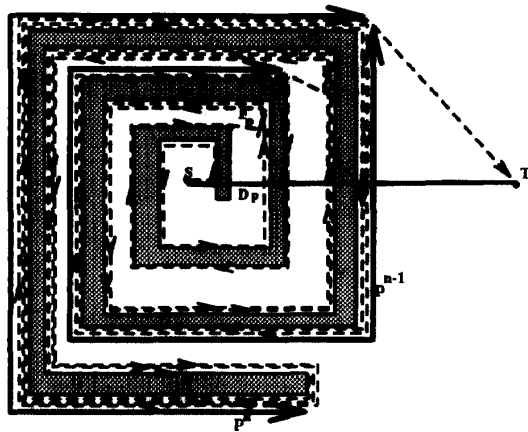


Fig. 7. The length of the path generated by *Angulus* algorithm in this example grows without bound: that is, the number of times a single path segment is traversed increases when more winds are added to this spiral.

lengths of segments of the line ST that are in free space and let P be the length of the path traversed. Since the free path segments are always longer than the corresponding pieces of ST (for example, $F_p > D_p$, Figure 7), they can be replaced accordingly in the inequalities below:

$$P > D + np^n + (n-1)p^{n-1} + \dots + p^1.$$

Choose p such that $p^n = p - \delta/n$. Then:

$$P > D + np - \delta + (n-1)p^{n-1} + \dots > D + np - \delta.$$

By sandwiching together a number of spiral obstacles, this inequality gives (1). *qed*.

There is the interesting question of whether algorithms with good finite performance bounds can be designed in the case of an angular input. Indeed they can: by assuming some more angular input information, the worst case bound can be reduced to, $2D + 2\sum p_i$, where p_i is the perimeter of each obstacle and D , the distance ST . Furthermore, this bound can most likely be reduced to $D + 2\sum p_i$ and this last conjecture constitutes work in progress. Finally, Figure 8 gives an example of the algorithm's performance, first for simple tactile sensing and then for vision sensing [12].

Acknowledgement: We would like to thank K. Hariharan for providing Example 2 and for writing the C-code for the simulations.

REFERENCES

- [1] O. Ore, *Theory of graphs*, Providence, RI: Amer. Math. Society, 1962.
- [2] C. Yap, Algorithmic motion planning. In *Advances in Robotics: I: Algorithmic and Geometric Issues*, (J. Schwartz and C. Yap, Eds). Hillsdale, NJ: L. Erlbaum Associates, 1987.
- [3] J.T. Schwartz, M. Sharir, On the "Piano Movers" Problem. II. General Techniques for Computing Topological Properties of Real Algebraic Manifolds. *Advances in Applied Mathematics*. Academic Press. No.4 (298-351), 1983.
- [4] V.J. Lumelsky, A. Stepanov, "Path Planning Strategies for a Point Mobile Automaton Moving Amidst Unknown Obstacles of Arbitrary Shape", *Algorithmica*, Springer-Verlag, 2, 1987, 403-430.
- [5] M. Blum, D. Kosen, On the Power of the Compass (or, Why Mazes are Easier to Search than Graphs). *Proc. of 19th Conf. on Foundations of Computer Science (FOCS)*, Ann Arbor, Michigan, 1978.
- [6] H. Abelson and A. diSessa, *Turtle Geometry*, MIT Press, Cambridge MA, 1980.
- [7] A. Sankaranarayanan, I. Masuda, "A New Algorithm for Robot Curve-Following Amidst Unknown Obstacles, and a Generalization of Maze-Searching", *Proc. of the 1992 IEEE International Conference on Robotics and Automation*, Nice, France, May 1992, pp 2487-2494.
- [8] V. Guillemin and A. Pollack, *Differential Topology*, Prentice-Hall, Englewood Cliffs N.J., 1974.
- [9] R. Courant, *Differential and Integral Calculus, Vol I*, Dover Publications, New York, 1964.
- [10] V. Lumelsky, S. Tiwari, New Algorithms and Performance Bounds for Maze Searching, University of Wisconsin Robotics Laboratory, Technical Report RL-94002, February 1994.
- [11] J. A. Thorpe, *Elementary Topics in Differential Geometry*, Springer-Verlag, New York, 1979.
- [12] T. Skewis, V.J. Lumelsky, "Computer Simulation of Sensor-Based Robot Motion", Yale University, Center for Systems Science, Department of Electrical Engineering, Tech. Report 9013, 1990.

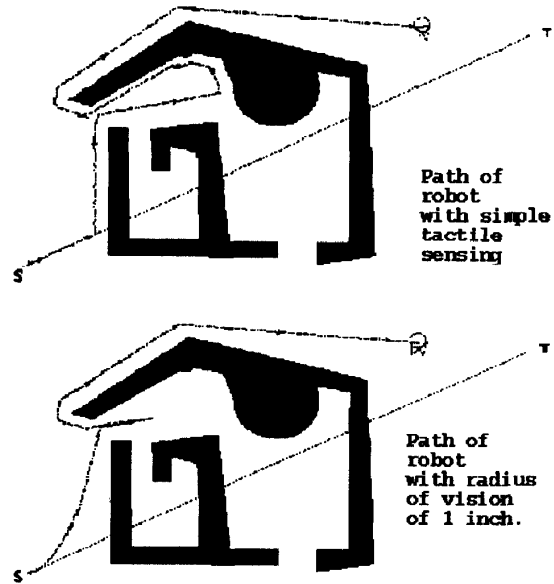


Fig. 8. The simulation of the path generated by the *Angulus* algorithm with and without vision.